

- **aio-max-nr**
系统支持的最大 **异步 I/O 请求数**。
如果使用 `libaio` 或 `io_uring`，这个值过小可能会限制并发请求。
- **aio-nr**
当前系统已分配的 **异步 I/O 请求数**（只读）。
- **dentry-state**
内核 **目录项缓存 (dentry cache)** 的状态，多个值组成，用于调优 VFS 缓存。
主要用于调试/分析，不建议直接改。
- **dir-notify-enable**
是否启用 `dnotify`，现在基本被 `inotify` / `fanotify` 取代。
默认 1（启用）。
- **epoll/max_user_watches**
单个用户 (UID) 能注册的 **epoll 监听项上限**。
影响 `inotify` 和 `epoll` 同时的资源占用，过小可能导致监控大量文件失败。
- **file-max**
系统允许的 **最大文件句柄数 (file descriptors)**，影响整个系统所有进程总和。
注意和 `/proc/sys/fs/nr_open`、`ulimit -n` (per-process limit) 区别。
- **file-nr**
三个值：
 - 已分配文件句柄数
 - 已使用文件句柄数
 - 系统文件句柄数上限（即 file-max）
- **inotify/max_queued_events**
`inotify` 事件队列的最大长度。队列满会丢事件。
- **inotify/max_user_instances**
单个用户允许创建的 `inotify` 实例数。
- **inotify/max_user_watches**
单个用户能监控的文件数上限。
如果写文件监控工具（如 `fswatch`、IDE 文件监控），这个值很关键。
- **leases-enable**
是否启用 **文件租约 (file lease)** 机制，用于进程间协调文件访问。
一般保持 1。
- **mount-max**
系统支持的最大挂载点数量。
- **mqueue/**
POSIX 消息队列相关配置，例如 `/proc/sys/fs/mqueue/msg_max`（每个队列消息数上限）。
- **nr_open**
单个进程可打开的最大文件描述符数量 (hard limit)。
比如设成 `1048576` 表示单进程最多开 1048576 个 FD。
`ulimit -n` 的上限受它限制。
- **overflowuid / overflowgid**
当 NFS 等文件系统无法映射远端用户 ID 时，用这个 UID/GID 代替。
默认 65534（即 nobody）。
- **pipe-max-size**
每个管道 (pipe) 的最大容量（字节数）。
默认值内核定义，改大可以减少写阻塞，但会占更多内存。
- **protected_hardlinks / protected_symlinks**
是否启用对 **硬链接/符号链接的安全保护**。
 - 1 = 开启保护（推荐，防本地提权漏洞）

- 0 = 关闭
- **suid_dumpable**
控制 setuid 程序崩溃时是否生成 core dump。
 - 0 = 不转储
 - 1 = 转储到调用者可写目录
 - 2 = 转储到安全位置 (/proc/sys/kernel/core_pattern 配置)

如何修改参数

- 临时修改（立即生效，重启丢失）：

```
echo 2097152 > /proc/sys/fs/file-max # 或 sysctl -w fs.file-max=2097152
```

- 永久修改：

```
echo "fs.file-max=2097152" >> /etc/sysctl.conf sysctl -p
```